



Gujarat Technological University



Government Engineering College, Sector-28, Gandhinagar

A Project Report on **Vehicle Accident Detection & Reporting System**

Under Subject of
Design Engineering-2B,
B.E. III, Semester VI,
Branch: Electronics & Communication

Submitted by: Godhrawala Murtaza - 230130111026

Group ID: 944813

Sr.	Name	Enrollment No.
1	Godhrawala Murtaza Mustafa	230130111026
2	Patel Prince Mukeshkumar	230130111083
3	Patel Vraj Ramanbhai	230130111087
4	Sarvaiya Bhautik Mukeshbhai	230130111115
5	Sikotara Kuldip Jatinbhai	230130111128

Prof. R.G. Pandey
(Internal Guide)

Dr. T.B. Sheth
(Head of Department)

Academic Year
(2025-2026)

Index

Sr. No.	Title	Page No.
1	Introduction	2
2	Modelling and Analysis using Software	4
3	Engineering Economics of Design	5
4	Design for Use, Reuse and Sustainability	7
5	Prototyping – Mark 1 and Mark 2	9
6	Testing the Prototype	12
7	Measuring Instruments and Techniques	14
8	Comparison of Materials, Methods and Tools	15
9	Conclusion and Future Scope	16
10	References	18

1. Introduction

1.1 Summary of Previous Semesters

This project, "Vehicle Accident Detection and Reporting System," was first conceptualized in Semester III as part of Design Engineering 1. During that phase, the team identified the critical problem of delayed emergency response after road accidents through ethnographic research tools such as the AEIOU Canvas, Empathy Canvas, and Mind Mapping Canvas. The problem statement was refined to target incapacitated drivers in remote or low-traffic areas who are unable to manually call for help.

In Semester IV (Design Engineering 1B), the team moved into ideation and early-stage prototyping, exploring different sensor combinations and communication strategies. The Ideation Canvas and Product Development Canvas were completed during this phase.

Semester V (Design Engineering 2A) marked the first physical implementation. An Arduino Uno was programmed with an MPU-6050 accelerometer, SW-420 vibration sensor, Neo-6M GPS module, and SIM800L GSM module. A Mark 1 prototype was assembled on a breadboard and tested on a temporary car chassis. While the detection logic, LCD alerts, and buzzer were functional, the SMS transmission could not be validated due to a defective SIM800L module. This gap was carried forward as the primary objective for Semester VI.

1.2 Objectives for Semester VI

Based on the limitations identified in Mark 1, the objectives for the current semester were defined as follows:

- Successfully validate SMS alert transmission with a verified and functional GSM module.
- Upgrade the microcontroller from Arduino Uno to ESP32 to gain improved processing power, built-in Wi-Fi, and larger flash memory.
- Integrate the Robot Operating System (ROS) framework for modular, node-based communication between system components.

- Address the SW-420 false-trigger problem by implementing a software debounce filter and a hardware cancel switch.
- Develop the Mark 2 prototype mounted on a rover chassis for realistic dynamic testing.
- Perform multi-scenario testing including head-on wall collision and multi-axis tilt simulations.

1.3 Scope of Work

The scope of this semester's work encompasses the complete redesign and upgrade of the embedded hardware, refinement of the detection algorithm, integration of a cancel-alert mechanism, and end-to-end testing of the SMS alert pipeline. The system remains a self-contained, standalone unit that operates without any smartphone companion application, relying solely on the cellular GSM network for emergency communication.

2. Modelling and Analysis using Software

2.1 Simulation Environment – Tinkercad

Prior to physical assembly, the updated circuit design for the Mark 2 prototype was modelled using Tinkercad Circuits. The ESP32 pinout was mapped with all peripherals including the MPU-6050 (I2C on GPIO 21/22), the SW-420 (GPIO 2), the buzzer (GPIO 8), the cancel switch (GPIO 15), the Neo-6M GPS (UART2), and the SIM800L GSM module (software serial). Tinkercad allowed the team to identify pin conflicts early, particularly the overlap between the default UART0 debug port and the SIM800L communication lines, which was resolved by assigning a dedicated software serial channel.

2.2 Programming Language

The ESP32 firmware was written in C using the ESP-IDF framework, which provides low-level hardware access and real-time task scheduling via FreeRTOS. Key advantages of using C over the Arduino abstraction layer include finer control over UART baud rates, interrupt service routines for the cancel switch, and the ability to create separate FreeRTOS tasks that map cleanly onto the ROS node concept. The ROS nodes were compiled and tested on a host PC running ROS Noetic (Ubuntu 20.04) before being ported to the ESP32 environment.

3. Engineering Economics of Design

3.1 Bill of Materials (BOM)

The following table presents the complete Bill of Materials for the Mark 2 prototype of the Vehicle Accident Detection and Reporting System:

Sr.	Component	Specification	Qty.	Cost (INR)
1	ESP32 Development Board	ESP32-WROOM-32	1	Rs. 300
2	SIM800L GSM Module	Quad-band GPRS	1	Rs. 500
3	Neo-6M GPS Module	u-blox NEO-6M	1	Rs. 500
4	MPU-6050 Accelerometer + Gyroscope	3-axis, I2C	1	Rs. 300
5	16x2 I2C LCD Display	I2C (0x27)	1	Carried from Mark 1
6	Buzzer	5V Active	1	Carried from Mark 1
7	Cancel Push Button Switch	Momentary, NO	1	Rs. 20
8	Breadboard, Wires, Resistors, Capacitors	Misc.	–	Rs. 180
9	Rover Chassis Kit (for testing)	4WD plastic	1	Carried forward
	TOTAL (new components this semester)			Rs. 1,800 approx.

3.2 Labour and Overhead Costs

As this is an academic prototype developed by students, no commercial labour cost is directly incurred. However, for a real-world cost estimation:

- Overhead (electricity, internet, laboratory space): Approximately Rs. 300 for the semester.

3.3 Total Estimated Cost

Cost Category	Amount (INR)
Hardware & Components	Rs. 1,800
Overhead	Rs. 300
Total	Rs. 2,100

4. Design for Use, Reuse and Sustainability

4.1 Design for Use

4.1.1 Reliability

The system is designed for continuous, unattended operation within a vehicle. The ESP32's dual-core architecture ensures that sensor polling, GPS parsing, and GSM communication can operate on separate FreeRTOS tasks without blocking each other. The MPU-6050's pre-calibrated threshold of 2.5g has been retained from Mark 1 and validated in Mark 2. A software debounce filter with a 200ms window is applied to the SW-420 to avoid spurious triggers from low-frequency motor vibration. The cancel switch provides an additional layer of human-in-the-loop reliability, ensuring that false alerts do not reach emergency contacts unnecessarily.

Expected operational lifespan of the electronic components under normal vehicle conditions: the ESP32 is rated for over 10 years at standard temperature ranges; the MPU-6050 has no moving parts and is highly durable; the SIM800L GSM module life depends on the SIM card's active subscription and network availability.

4.2 Design for Reuse

The codebase has been structured so that the ROS nodes can be repurposed for other IoT safety applications. The hardware stack (ESP32 + GPS + GSM) is a commonly used combination in IoT projects, meaning the skills and configurations developed here directly transfer to future projects.

The rover chassis used for testing can be repurposed as a robotics platform for future Design Engineering projects or laboratory experiments.

4.3 Design for Sustainability

The system consumes approximately 500mA on average, with peak current of up to 2A during GSM transmission. Powering the unit from the vehicle's 12V supply through a buck converter means no additional battery waste is generated. The

long-term sustainability of the system depends on:

- Component longevity: Solid-state electronics have no mechanical wear. The system can function for the lifetime of the vehicle with minimal maintenance.
- Software updatability: OTA (Over-the-Air) firmware updates via the ESP32's built-in Wi-Fi module can extend the system's functional life without physical access.
- Environmental impact: The BOM contains no hazardous materials beyond standard RoHS-compliant electronics. At end-of-life, components should be recycled through certified e-waste channels.
- Cost of operation: The only recurring cost is the GSM SIM card's data plan, which is negligible (a basic SMS plan costs less than Rs. 50/month).

5. Prototyping – Mark 1 and Mark 2

5.1 Mark 1 Prototype (Semester V – Previous Iteration)

The Mark 1 prototype was the first physical implementation of the system, developed during Semester V. It used an Arduino Uno as the central microcontroller and was assembled on a breadboard mounted to a temporary car chassis. Key findings from Mark 1 were:

- The MPU-6050 accelerometer successfully detected simulated crash events (wall impact and angular tilt beyond threshold).
- The SW-420 vibration sensor triggered correctly for high-impact events but also generated false positives when the DC motors driving the chassis started up, due to the sudden electrical and mechanical jerk.
- The LCD and buzzer alert pipeline was fully functional and provided immediate local feedback within 1 second of detection.
- SMS transmission could not be validated because the SIM800L module available to the team was found to be defective; it failed to register on the GSM network despite correct AT command sequences.
- No cancel mechanism existed in Mark 1; once an accident was detected, the alert sequence could not be aborted.

Mark 1 therefore established the proof-of-concept for detection and local alerting, but left the end-to-end SMS pipeline and false-detection mitigation as open problems to be addressed in Mark 2.

5.2 Mark 2 Prototype – Design Changes and Iterations

5.2.1 Iteration 1: Microcontroller Upgrade – Arduino Uno to ESP32

The first major design iteration was replacing the Arduino Uno with an ESP32-WROOM-32 development board. The motivation for this change was threefold. First, the ESP32 provides significantly more program flash (4MB vs 32KB on Arduino Uno), allowing the full ROS-based firmware to be stored. Second, the ESP32's dual UART peripherals allow simultaneous hardware serial communication with both the GPS module and the GSM module, eliminating the

software serial bottleneck that was identified as a potential source of timing conflicts in Mark 1. Third, the ESP32's built-in Wi-Fi and Bluetooth capabilities open a pathway for future IoT integration without any hardware changes.

The pin mapping was revised as follows:

Component	Arduino Uno Pin	ESP32 Pin
MPU-6050 SDA	A4	GPIO 21
MPU-6050 SCL	A5	GPIO 22
SW-420	D2	GPIO 2
Buzzer	D8	GPIO 8
Neo-6M GPS (TX/RX)	D3/D4 (SoftSerial)	UART2 – GPIO 16/17
SIM800L GSM (TX/RX)	D6/D7 (SoftSerial)	UART1 – GPIO 26/27
Cancel Switch	Not present	GPIO 15

5.2.2 Iteration 2: ROS Integration

The second design iteration introduced the ROS (Robot Operating System) middleware layer. Each functional subsystem of the vehicle accident detection system was encapsulated as an independent ROS node. The nodes communicate through the ROS publish-subscribe messaging model. This made the system easier to debug, as each node could be tested in isolation on a development PC before being deployed to the ESP32. The inter-node communication is handled via ROS topics, and the roslaunch file allows the entire system to start up with a single command.

5.3 Final Mark 2 Prototype Description

The Mark 2 prototype is assembled on a 4-wheel-drive rover chassis. The ESP32 development board is mounted centrally on a breadboard fixed to the chassis

deck. The MPU-6050 is positioned at the geometric centre of the chassis for optimal vibration and acceleration measurement. The Neo-6M GPS antenna is positioned at the top of the chassis with a clear upward line of sight for satellite acquisition. The SIM800L GSM module with its coil antenna is mounted at the rear of the chassis. The 16x2 I2C LCD is mounted at the front for visibility during testing, and the buzzer and cancel switch are accessible from the top surface of the chassis.

6. Testing the Prototype

6.1 Testing Methodology

Testing was conducted across three distinct scenarios using the rover chassis as a stand-in for a real vehicle. All tests were performed in an open-sky environment (outdoors or near a window) to ensure reliable GPS satellite lock. The GSM module was tested with a valid active SIM card.

6.2 Test Scenarios

Test 1: Head-On Wall Collision

The rover was driven at full speed into a solid brick wall. The sudden deceleration generated an impulse force exceeding the 2.5g threshold on the MPU-6050's X-axis, while simultaneously triggering the SW-420 due to the physical impact shock. The system correctly identified this as an accident event, activated the buzzer and LCD alert, and transmitted an SMS with a Google Maps location link within approximately 4 seconds of impact. No false cancel was pressed; the full alert pipeline was validated end-to-end.

Test 2: Multi-Axis Tilt / Roll Simulation

The rover was manually tilted beyond 90 degrees on its X, Y, and Z axes sequentially to simulate a vehicle rollover. The MPU-6050's gyroscope data registered angular rates exceeding the programmed threshold, triggering the detection logic. The SMS was successfully transmitted. This test validated the gyroscope-based detection arm of the sensor fusion logic.

Test 3: Cancel Switch Validation (False Positive Scenario)

The rover was gently nudged by hand, generating a vibration level that, in Mark 1, would have caused the SW-420 to trigger alone. With the dual-sensor AND logic in Mark 2, the nudge did not trigger an alert because the MPU-6050 did not exceed 2.5g. In a second sub-test, the team manually triggered both sensors simultaneously and then pressed the cancel switch within the 10-second grace window. The SMS was correctly suppressed, and the LCD displayed a "Alert

Cancelled" message.

6.3 Results Summary

Test Scenario	Detection	SMS Sent	Response Time
Head-on wall collision	Successful	Yes	~4 seconds
Rollover (tilt simulation)	Successful	Yes	~3.5 seconds
Motor startup jerk (false positive)	Not triggered (correct)	No	N/A
Cancel switch during grace period	Detected, then cancelled	Suppressed	N/A

7. Measuring Instruments and Techniques

7.1 Digital Multimeter

A digital multimeter (DMM) was the primary measurement instrument used during the development and debugging of the Mark 2 prototype. The following measurements were taken:

- **ESP32 GPIO Voltage Levels:** Digital output pins of the ESP32 were probed to verify that HIGH output was 3.3V and LOW was 0V, confirming proper logic levels for sensor input lines.
- **MPU-6050 Power Line:** The VCC pin of the MPU-6050 was confirmed to be receiving 3.3V from the ESP32's regulated output, ensuring the sensor operated within its specified voltage range.
- **Current Draw Measurement:** By placing the multimeter in series with the power supply line, the idle current draw was measured at approximately 180mA, rising to approximately 1.8A peak during GSM transmission.

7.2 Arduino Serial Monitor / ROS Logger

During software debugging, the Arduino IDE Serial Monitor (and subsequently the ROS `rqt_console`) was used as a virtual measurement instrument to observe real-time sensor data. The raw acceleration values from the MPU-6050 were logged to identify the correct threshold value (2.5g) that distinguishes genuine impact events from normal driving vibrations. The GPS NMEA sentence output was monitored to verify satellite lock acquisition and coordinate accuracy.

8. Comparison of Materials, Methods and Tools

8.1 Arduino Uno vs ESP32

The most significant hardware design decision in this semester was the migration from the Arduino Uno to the ESP32. The following table presents a structured comparison that justifies this selection:

Parameter	Arduino Uno (ATmega328P)	ESP32-WROOM-32
Clock Speed	16 MHz (single core)	240 MHz (dual core)
Flash Memory	32 KB	4 MB
RAM	2 KB SRAM	520 KB SRAM
UART Ports	1 hardware UART	3 hardware UARTs
Wi-Fi / Bluetooth	Not available	Built-in (802.11 b/g/n + BT 4.2)
ROS Compatibility	Limited (rosserial only)	Full ROS via micro-ROS / ESP-IDF
Real-Time OS	No native RTOS	FreeRTOS built-in
Cost	Rs. 350 (approx.)	Rs. 300 (approx.)
Development Environment	Arduino IDE	Arduino IDE / ESP-IDF / VS Code
Verdict	Adequate for simple, single-task embedded systems	Superior for multi-task, IoT-connected, and ROS-based systems

The ESP32 was selected because it satisfies all the requirements of this project at a lower or equal cost, while providing significant headroom for future enhancements. The additional UART ports alone resolved the software serial timing conflict that existed in Mark 1.

9. Conclusion and Future Scope

9.1 Conclusion

This project, developed across four semesters of Design Engineering, has successfully culminated in a fully functional Mark 2 prototype of the Vehicle Accident Detection and Reporting System. The system reliably detects vehicle accidents through dual-sensor fusion of the MPU-6050 accelerometer and SW-420 vibration sensor, captures real-time GPS coordinates via the Neo-6M module, and transmits an SMS alert with a Google Maps location link to a pre-configured emergency contact via the SIM800L GSM module, all within 3 to 5 seconds of incident occurrence.

The Semester VI iteration introduced a significant microcontroller upgrade from Arduino Uno to ESP32, enabling true multi-task execution through FreeRTOS, native hardware UART channels for GPS and GSM communication, and ROS integration for modular and maintainable software architecture. The false-detection problem identified in Mark 1 was resolved through dual-sensor AND logic, a 200ms software debounce filter, and the introduction of a manual cancel switch with a 10-second grace period.

The complete system was tested on a rover chassis under three simulated accident scenarios — head-on wall collision, rollover tilt, and false-positive suppression — and passed all test cases successfully. The prototype demonstrates that a highly effective vehicle safety system can be built for approximately Rs. 1,800 in hardware costs, making it a viable candidate for widespread adoption in affordable vehicle segments.

9.2 Future Scope

Several enhancements can be pursued in future work to transition this prototype into a production-ready system:

- **PCB Fabrication:** Designing and fabricating a custom Printed Circuit Board to house all components in a compact, vibration-resistant form factor suitable for permanent vehicle installation. This was the primary goal not

achieved in this semester due to time and resource constraints.

- IoT Cloud Integration: Leveraging the ESP32's built-in Wi-Fi to transmit accident data to a cloud platform such as ThingSpeak, AWS IoT, or Google Firebase, enabling real-time monitoring dashboards and historical accident data analysis.
- Emergency Voice Call: Extending the SIM800L's functionality to place an automated voice call to emergency services (such as 108 or 112) in addition to the SMS alert, enabling two-way communication for rescue coordination.
- Engine Cutoff Mechanism: Integrating a relay module controlled by the ESP32 to automatically cut the vehicle's ignition after a severe accident is detected, preventing fuel leakage and post-crash fires.
- Advanced ML-Based Detection: Implementing a lightweight machine learning model (using TensorFlow Lite for Microcontrollers on the ESP32) trained on accelerometer data to distinguish genuine crash signatures from road bumps and aggressive driving, further reducing false positives.
- Mobile App Companion: Developing a smartphone application that receives real-time alerts via the ESP32's Bluetooth or through Firebase push notifications, providing a richer user experience than SMS alone.
- Alcohol Detection Integration: Adding an MQ-3 alcohol sensor to the system to detect driver intoxication and send a pre-emptive alert before an accident occurs, as envisioned in the original block diagram concept.

10. References

1. International Journal of Engineering Research & Technology (IJERT) – "Automatic Vehicle Accident Detection and Messaging System": <https://www.ijert.org/automatic-vehicle-accident-detection-and-messaeing-system>
2. Hash Studios Technologies – "IoT-based Automatic Vehicle Accident Detection and Rescue System": <https://hashstudioz.com/blog/iot-based-automatic-vehicle-accident-detection-and-rescue-system/>
3. XiLir Technologies – "IoT Vehicle Accident Detection Alert System Using Arduino" [YouTube]: <https://youtu.be/76cptRVjsiA>
4. svseembeddeb – "Vehicle Accident Prevention & Reporting System Using GSM + GPS + MEMS and Arduino" [YouTube]: <https://youtu.be/uOUURhb0eEM>
5. Espressif Systems – ESP32 Technical Reference Manual: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
6. ROS Wiki – ROS Noetic Documentation: <https://wiki.ros.org/noetic>
7. u-blox – NEO-6M GPS Module Datasheet: https://www.u-blox.com/sites/default/files/products/documents/NEO-6_DataSheet.pdf
8. InvenSense – MPU-6050 Product Specification: <https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet1.pdf>
9. SIMCOM – SIM800L AT Commands Manual: https://simcom.ee/documents/SIM800L/SIM800%20Series_AT%20Command%20Manual_V1.09.pdf
10. ChatGPT (OpenAI): <https://chatgpt.com> – Used for debugging AT command sequences and ROS node structure guidance.